

PRODUCT REQUIREMENTS DOCUMENT

Local Library Attendance & Study-Space Management System

PRD Version 1.0 • Offline-First MVP

Purpose: A cost-effective desktop system for local study libraries to manage student registration, seat allocation, QR-based attendance, attendance history, reports, export, and data backup.

Prepared as a professional product blueprint for validation, development and future scaling.

Document Control

Item	Details
Product	Local Library Attendance & Study-Space Management System
Version	1.0
Architecture direction	Offline-first, modular, cloud-ready
Initial deployment	Single library / single Windows computer
Primary interface	Desktop application
Primary attendance method	Webcam QR scanning
Primary storage	SQLite local database
Export	CSV and Excel
Future direction	Multi-library, cloud sync, web/mobile dashboards

Version history

v1.0 — Initial product definition covering MVP scope, workflow, architecture, data model, security, testing, deployment and scalability strategy.

1. Executive Summary

Product Vision

Create a simple, affordable and reliable attendance system for local study libraries where students can check in using a personal QR code at a dedicated computer/webcam station. The system records the student identity, seat and exact scan timestamp without requiring expensive biometric hardware or continuous internet.

Core principle

The first release should prove value before adding complexity. Build offline-first, keep hardware requirements minimal, make daily operation nearly one-click, and design the data layer so cloud synchronization can be introduced later.

Business outcome

Replace paper/register-based attendance with searchable digital records, quick daily reporting, exportable data and a defensible entry/attendance history.

2. Problem Statement

Current pain points

Local libraries commonly depend on manual registers or spreadsheets. This can be slow, difficult to search, prone to handwriting/data-entry errors, and inconvenient when an owner needs historical attendance records.

Operational risk

When an incident or dispute occurs, the owner may need to establish whether a student was recorded as present and at what time. The product should therefore preserve timestamped attendance events and avoid silently overwriting historical records.

Constraints

Owners may not want subscriptions, specialized scanners, biometric devices, dedicated servers or complicated setup. The MVP must work with an ordinary Windows PC and a low-cost USB webcam.

3. Goals & Success Criteria

Goals

1) Record attendance in seconds. 2) Reduce manual entry. 3) Maintain searchable historical records. 4) Export records for future use. 5) Require minimal hardware. 6) Work without internet for normal operation. 7) Establish a foundation for later multi-library/cloud deployment.

MVP success criteria

A trained operator can start the application, open the scanner, scan a valid student QR, see the student's name/seat and timestamp, and save the event without typing student details.

4. Scope

In scope — MVP

Admin login; student registration; unique Student ID; seat allocation; QR generation/printing or saving; webcam scanner; attendance event creation; duplicate-scan protection; check-in/check-out policy; attendance history; date/student/seat filters; CSV/Excel export; database backup/restore; basic settings; error messages; Windows deployment.

Out of scope — MVP

Cloud hosting; mobile application; biometric attendance; facial recognition; online student portal; automated fee collection; advanced analytics; CCTV integration; multi-branch administration.

Important design decision

Student registration remains necessary once per student. The QR should ideally contain only a non-sensitive unique Student ID rather than the student's full personal data. The scanner uses that ID to retrieve current name/seat details from the local database.

5. Users & Roles

Library Owner / Admin

Registers students, assigns seats, manages records, views reports, exports data, performs backups and configures the system.

Staff — optional

Can operate scanner and view limited attendance functions if the owner later enables staff accounts.

Student

Receives a unique QR code and presents it at the attendance station. The student does not need access to the admin application.

6. Core User Workflow

Initial setup

Install application → create owner account → configure library details → create seats → register students → assign seats → generate student QR cards.

Daily attendance

Start application → webcam initializes → student presents QR → QR decoded → Student ID validated → name and seat displayed → timestamp generated by computer → attendance event stored → success indicator → scanner immediately waits for next QR.

Record review

Owner opens attendance/report screen → selects date range/student/seat/status → reviews events → exports CSV/Excel if required.

Backup

Owner runs backup manually on a schedule; system copies the database to a selected backup location. Restore should be available from the settings/admin area.

7. Functional Requirements

FR-01 Student Management

Create, view, edit, deactivate and search student records. Minimum fields: Student ID, full name, phone (optional), seat number, joining date, status and notes (optional).

FR-02 Seat Management

Create seats and mark them available/occupied/inactive. Prevent two active students from holding the same seat unless the owner explicitly allows it.

FR-03 QR Management

Generate a unique QR per student. QR payload should be a random/unique Student ID or token. Provide print/save capability.

FR-04 Scanner

Use a webcam to continuously detect QR codes. Show camera status, last scanned student, seat and timestamp. Return to scanning state automatically.

FR-05 Attendance

Store student ID, event date/time, event type/status, seat at scan time, device/application metadata where appropriate, and a unique event ID.

FR-06 Duplicate protection

Prevent accidental repeated scans within a configurable short interval while still allowing legitimate later attendance events.

FR-07 Reports

Today, date range, student-wise, seat-wise and attendance-status reports. Provide counts and detailed rows.

FR-08 Export

Export filtered report results to CSV and Excel (.xlsx). Preserve date/time values and clear column headings.

FR-09 Backup/Restore

Create database backups and restore from a selected valid backup. Require admin confirmation before restore.

FR-10 Settings

Library name, scanner settings, duplicate-scan interval, attendance policy, backup location and basic application preferences.

FR-11 Auditability

Do not edit historical attendance records through normal student-edit operations. Corrections should be represented as controlled actions or clearly logged changes.

8. Attendance Policy

Recommended MVP rule

Use explicit Check-in and Check-out events. A student's first valid scan after the configured opening threshold can create Check-in; a subsequent valid scan can create Check-out, subject to configurable rules.

Alternative

If the library only needs presence records, use a single 'Attendance' event per day and block duplicate scans for the same student/day. The PRD should keep the event model flexible so this can change without redesigning the database.

Timestamp

Use the local computer clock for MVP. Store timestamps in a consistent database format and display them in the library's local timezone.

9. Non-Functional Requirements

Performance

Scanner UI should feel immediate after a successful QR read. Database operations should be lightweight for a single-library workload.

Reliability

A temporary webcam failure should not crash the application. The app should clearly show camera/database status.

Offline-first

Normal registration, scanning, reporting and export should not require internet.

Usability

Large scanner status, clear success/error messages, minimal clicks and simple navigation for non-technical owners.

Security

Admin authentication; local database access controls appropriate to a desktop application; minimize sensitive data in QR payload; protect backups; avoid exposing student data unnecessarily.

Maintainability

Separate UI, business logic, QR/scanner services, database/repository, reporting and configuration modules.

Scalability

Keep Student IDs and attendance events independent from UI; use repository/service interfaces so SQLite can later be replaced or complemented by a server database.

10. Recommended Tech Stack

Language

Python 3.11/3.12 (choose one stable supported version for the project and pin dependencies).

GUI

CustomTkinter or Tkinter for a lightweight Windows desktop interface. CustomTkinter is preferred if a modern appearance is desired.

QR scanning

OpenCV plus a QR decoding library such as OpenCV's QRCodeDetector or pyzbar, depending on Windows dependency constraints. Start with the simplest reliable option.

Database

SQLite for MVP; accessed through Python's sqlite3 module or SQLAlchemy if an abstraction layer is desired.

Data/reporting

Pandas for report processing; openpyxl for .xlsx export.

Packaging

PyInstaller to create a Windows executable.

Development

VS Code, Git and GitHub.

Testing

pytest for automated tests; manual webcam/QR tests for hardware integration.

Architecture recommendation

Desktop UI → application/service layer → repository/data layer → SQLite. QR scanner and export modules remain separate services.

11. High-Level Architecture

Flow

Webcam → QR Decoder → Attendance Service → Student/Seat Repository → SQLite → Reports/Export.

Why this architecture

It keeps hardware handling separate from business rules. Later, a web API can call the same application/business logic patterns while a cloud database can replace the local repository.

Internet dependency

None for the core MVP. Internet may be required only for installation/update workflows unless a future cloud sync feature is enabled.

12. Database Design

students

student_id (PK), full_name, phone, joining_date, status, created_at, updated_at, notes.

seats

seat_id (PK), seat_number, status, created_at, updated_at.

attendance_events

event_id (PK), student_id (FK), seat_id/seat_number snapshot, event_type, event_timestamp, created_at, device_id (optional), remarks.

admins

admin_id (PK), username, password_hash, created_at, status.

settings

key, value, updated_at.

audit_log — recommended

log_id, admin_id, action, entity_type, entity_id, timestamp, details.

Relationship

One student can have many attendance events. One seat can be associated with many students over time but should normally have only one active occupant. Historical attendance should retain the seat snapshot so later seat changes do not rewrite history.

13. QR Code Design

Recommended payload

A non-guessable Student ID/token, for example a generated identifier such as LIB-8F4K2M. Do not embed phone number, address or other unnecessary personal information.

Why not store name and seat directly in QR

If a student changes seats or their name/details are corrected, an old QR would become stale. A stable identifier lets the database remain the source of truth.

Student experience

Student receives/prints the QR once. Daily attendance requires only presenting it to the webcam.

14. UI / Screen Plan

Login

Username/password, library branding, basic status.

Dashboard

Today's attendance count, current active students, scanner status, quick actions.

Student Management

Search, add/edit/deactivate student, assign seat, generate QR.

Scanner

Large camera preview, scan status, last student, seat, timestamp, error/success notification.

Attendance

Filterable event table and details.

Reports

Date range, student, seat, status filters; totals; export CSV/Excel.

Seats

Visual/list view of available and occupied seats.

Settings

Library info, attendance policy, camera, duplicate interval, backup/restore.

15. Error Handling

Camera unavailable

Show a clear message and allow retry/reselect camera.

Invalid QR

Show 'QR not recognized' without creating an attendance record.

Unknown Student ID

Show 'Student not registered' and log the failed scan only if desired.

Duplicate scan

Show a non-destructive warning and do not create an accidental duplicate event.

Database failure

Stop writing operations safely, show an actionable message and avoid data corruption.

Export failure

Explain that the destination file may be open or inaccessible and provide retry/change-location guidance.

Unexpected shutdown

SQLite transaction handling should prevent partially written attendance events. On restart, the system should recover normally.

16. Testing Strategy

Unit tests

Student validation, seat allocation rules, duplicate-scan logic, attendance event creation, report filtering and export data.

Integration tests

QR decoder → student lookup → attendance database; backup/restore; Excel export.

Hardware tests

Different webcam resolutions/lighting conditions, printed QR sizes, screen-displayed QR, multiple consecutive scans.

Acceptance tests

A non-technical library operator should be able to install, register a student, generate QR, scan attendance and export a daily report using the documented workflow.

17. Deployment Plan

MVP deployment

Windows PC + USB webcam. Package application as an EXE with PyInstaller. Include a first-run setup wizard.

Installation

Install/copy application → connect webcam → launch → create admin → configure library → create seats → register students → generate QR cards → test scan.

Data location

Keep application data in a predictable user-writable directory, separate from the executable. Do not store the database inside a temporary extraction directory.

Updates

For MVP, manual versioned EXE updates are acceptable. Future versions can introduce an update mechanism.

18. Backup & Recovery

Recommended

Automatic daily local backup plus a manual 'Backup Now' button. Keep multiple dated backup copies.

Recovery

Admin selects a backup → application validates it → confirmation → creates a safety copy of current database → restores selected backup → restarts/reloads data.

Operational recommendation

For important records, copy backups to an external drive or another secure location. Offline-first does not mean backup-free.

19. Cost Strategy

Hardware

Reuse an existing Windows desktop/laptop and purchase only a basic USB webcam if needed.

Software

Use Python and open-source libraries; avoid paid attendance hardware and monthly SaaS fees for the MVP.

Hosting

■0 recurring hosting cost for the offline MVP. Cloud costs are deferred until there is a business reason for synchronization or remote access.

Commercial strategy

First prove the system in one library. Pricing can later be based on one-time installation/license, optional support, or a cloud subscription for advanced features.

20. Scalability Roadmap

Phase 1 — MVP

Single library, one Windows machine, SQLite, webcam QR attendance, reports, Excel/CSV export, backup.

Phase 2

Better dashboard, staff accounts, richer audit log, automatic backups, improved seat/attendance policies, installer/update improvements.

Phase 3

Cloud backend/API, secure authentication, multi-device operation, remote owner dashboard and optional cloud backup.

Phase 4

Multi-library/branch management, subscription/license management, mobile app, analytics, fee management and integrations.

Key principle

Do not build cloud infrastructure before users need it. Design clean interfaces now so migration later is possible without rewriting the entire product.

21. Risks & Mitigations

QR sharing

A student could share their QR. Mitigation: use check-in/out logic, duplicate/time rules, optional staff verification, and future device/location controls.

Camera/lighting issues

Use a good-enough webcam position and clear QR cards; provide camera test screen.

Data loss

Automated and manual backups.

Privacy

Collect only necessary student data; keep QR payload minimal; protect exported files.

Owner adoption

Keep daily workflow extremely simple and provide a short printed/visual operating guide.

Scope creep

Freeze MVP features before coding; move fees, cloud, mobile and advanced analytics to later phases.

22. Open Product Decisions

Questions to validate with real library owners

Do they need only daily presence or both check-in/check-out? Do they already assign permanent seats? Do students carry printed QR cards or prefer phone QR? How often are reports required? Do they need phone numbers? What retention period is expected? Who is allowed to export records?

Validation rule

These questions should be answered by interviewing at least a few local library owners before finalizing the production version. The PRD can then be revised to v1.1.

23. MVP Definition of Done

The MVP is ready for pilot use when

A Windows computer can run the application reliably; webcam scanning works; students can be registered and assigned seats; unique QR codes can be generated; scans create timestamped attendance events; duplicates are controlled; reports can be filtered; CSV and Excel export works; backup/restore works; errors are understandable; and a non-technical owner can operate the daily scanner flow after brief training.

24. Development Plan

Step 1

Freeze requirements and validate workflow with real owners.

Step 2

Create repository, Python environment and dependency lock/requirements file.

Step 3

Implement database schema and repository layer.

Step 4

Implement student/seat management.

Step 5

Implement QR generation.

Step 6

Implement webcam QR scanning and attendance service.

Step 7

Build dashboard and reports.

Step 8

Add CSV/Excel export and backup/restore.

Step 9

Testing, bug fixing and pilot deployment.

Step 10

Collect owner feedback and release v1.1.

25. AI-Assisted Development Guidelines

How to use coding AI effectively

Use AI as a pair programmer, not as an uncontrolled code generator. Give it the PRD section, exact task, existing folder structure and constraints. Ask for one module at a time, then run and test before moving on.

Recommended prompt pattern

Context → Goal → Constraints → Existing files/code → Expected behavior → Acceptance criteria → Ask for explanation and tests.

Quality rule

Do not accept generated code merely because it runs. Review dependencies, security, database transactions, error handling and tests. Keep commits small so changes can be reverted.

26. Final Product Positioning

One-line description

An affordable offline-first QR attendance system for local study libraries that turns a webcam and an existing computer into a simple, searchable attendance station.

Value proposition

Minimal hardware + no mandatory internet + fast QR attendance + exportable records + future cloud scalability.